AI Development Agent (ADA) — Complete Project Specification and Implementation Prompt

Project Overview

Build a Python-based AI Development Agent (ADA) that acts as a controlled software engineering system capable of analyzing, planning, generating, modifying, validating, testing, reviewing, and managing software projects.

The objective is NOT to create another "vibe coding" tool, code editor extension, or simple AI wrapper.

The objective is to create a system where:

AI provides reasoning and software design decisions.

Python controls execution, validation, safety, filesystem operations, testing, and project management.

The system must prioritize:

1. Cost Reduction
2. Quality Assurance
3. Automation

in that exact order.

Core Philosophy

The system must follow the principle:

> AI proposes.
>
> Python controls.
>
> Python validates.
>
> AI reasons about failures.
>
> Nothing reaches the real project until validation succeeds.

The AI must never directly control the user's computer.

The AI must never directly execute commands.

The AI must never directly write files.

The AI only produces structured instructions.

Python remains the authority responsible for:

- Reading files
- Writing files
- Creating directories
- Applying patches
- Running tests
- Running validations
- Managing Git
- Creating sandboxes
- Enforcing permissions
- Approving or rejecting AI operations

The AI is a reasoning engine, not an operating system.

Primary Goal

Create a local AI software engineering environment that can:

- Understand existing projects
- Analyze project architecture
- Generate new features
- Modify existing code
- Debug failures
- Run validations
- Review generated code
- Create project structures
- Manage multiple programming languages
- Operate safely through a sandbox system
- Reduce AI API cost through intelligent context management

The system should eventually support:

- Django
- Generic Python
- FastAPI
- Flask
- Node.js
- React
- Vue
- Laravel

while keeping the core architecture framework-independent.

Strategic Objectives

Objective 1: Cost Reduction

The system must minimize AI API usage.

AI should only be used for tasks requiring reasoning.

Python should perform all deterministic operations.

Examples:

Python handles:

- Reading files
- Searching files
- Project scanning
- Directory creation
- File creation
- Applying patches
- Running tests
- Running linting
- Running type checking
- Running Django checks
- Running Git commands
- Capturing errors

AI handles:

- Planning
- Architecture decisions
- Code generation
- Error diagnosis
- Feature design
- Refactoring decisions
- Code review

The architecture must prevent wasting tokens on operations Python can perform directly.

Objective 2: Quality

The system must prioritize correctness over speed.

Generated code must pass validation before being accepted.

Validation should include:

- Syntax validation

- Import validation
- Framework validation
- Test execution
- Type checking
- Linting
- Build validation

The system must reject invalid changes.

Objective 3: Automation

The system should gradually evolve toward autonomous software engineering.

Target workflow:

User request → Analysis → Planning → Code generation → Sandbox application → Validation → Debugging → Revalidation → Review → Approval → Real project

---

System Architecture

Create the following architecture.

```
ai_dev_agent/

├── ai/
├── agent/
├── project/
├── context/
├── filesystem/
├── sandbox/
├── validation/
├── git/
├── history/
├── config/
├── cli/
└── main.py
```

Module Specifications

ai/

Purpose:

AI provider abstraction layer.

Requirements:

- Provider-independent design
- Easy model switching
- Structured responses only

Supported providers:

- OpenAI
- Anthropic
- Gemini
- Local Models

Design:

```
class AIProvider:
    def generate(...)
```

The rest of the system must never depend on a specific AI provider.

---

agent/

Purpose:

Workflow orchestration.

Modules:

```
planner.py
coder.py
reviewer.py
debugger.py
orchestrator.py
```

Responsibilities:

Planner:

- Understand user requests
- Determine affected areas
- Create implementation plans

Coder:

- Generate implementation operations

Reviewer:

- Review generated code
- Detect potential design issues

Debugger:

- Analyze validation failures
- Propose corrections

Orchestrator:

- Coordinate the complete workflow

This becomes the central brain of the application.

---

project/

Purpose:

Project understanding.

Responsibilities:

- Detect framework
- Analyze structure
- Map dependencies
- Discover applications/modules
- Understand architecture

Must detect:

- Django
- FastAPI
- Flask
- Generic Python
- Node.js
- React

Examples:

Django detection:

- manage.py
- settings.py
- INSTALLED_APPS
- urls.py

Outputs:

Project metadata and structure map.

---

context/

Purpose:

Cost optimization through intelligent context selection.

This is one of the most important modules.

Requirements:

Never send an entire project to the AI.

Instead:

User Request → Determine relevance → Extract relevant files → Extract relevant sections → Build minimal context → Send only required information

Modules:

```
collector.py
relevance.py
builder.py
cache.py
tokenizer.py
```

Version 1:

Use:

- filenames
- imports
- references
- project structure
- keyword search

Avoid embeddings initially.

Embeddings and vector search can be added later.

---

filesystem/

Purpose:

Controlled file operations.

Responsibilities:

```
create_file()
modify_file()
delete_file()
create_directory()
read_file()
apply_patch()
```

The AI must never directly call these functions.

AI returns operations.

Python validates them first.

---

sandbox/

Purpose:

Protect the real project.

Rules:

AI modifications must never be applied directly to the production project.

Workflow:

Real Project → Create Sandbox Copy → Apply AI Changes → Validate → Review → Approve → Apply To Real Project

Failed modifications:

- Destroy sandbox
- Keep project untouched

Modules:

```
manager.py
workspace.py
cleanup.py
```

Sandbox is mandatory.

---

validation/

Purpose:

Quality assurance.

Responsibilities:

- Syntax checking
- Framework validation
- Linting
- Type checking
- Test execution

Modules:

```
syntax.py
tests.py
framework.py
lint.py
types.py
pipeline.py
```

Examples:

Python:

```
python -m compileall .
pytest
ruff check .
mypy .
```

Django:

```
python manage.py check
python manage.py test
python manage.py makemigrations --check
```

Validation results must be standardized.

Example:

```
ValidationResult(
    success=False,
    errors=[],
```

```
    warnings=[]
)
```

---

git/

Purpose:

Version control and safety.

Responsibilities:

- status
- diff
- rollback
- branch management
- commit management

Requirements:

Every change must be traceable.

Workflow:

Before changes: git status

After changes: git diff

Before final application: show diff

After approval: commit

Rollback capability is mandatory.

---

history/

Purpose:

Audit and debugging.

Store:

- user requests
- AI prompts
- AI responses
- generated operations
- validation results

- diffs
- corrections

Purpose:

Full reproducibility of agent decisions.

AI Communication Rules

The AI must never return free-form coding instructions.

All responses must use structured operations.

Example:

```
{
  "operations": [
    {
      "action": "create_file",
      "path": "notifications/models.py",
      "content": "..."
    }
  ]
}
```

Allowed operation types:

```
CREATE_FILE
MODIFY_FILE
DELETE_FILE
CREATE_DIRECTORY
APPLY_PATCH
```

Optional:

```
RUN_ALLOWED_COMMAND
```

Only if approved by command policy.

Command Execution Policy

The AI must not execute arbitrary commands.

Allowed:

```
python
pytest
ruff
mypy
git
django-admin
manage.py
```

Restricted:

```
pip install
npm install
```

Forbidden:

```
sudo
rm -rf
system configuration
disk formatting
network administration
OS modification
```

Python must enforce command policies.

Task State Machine

Implement explicit workflow states.

```
IDLE

ANALYZING

PLANNING

CONTEXT_BUILDING

CODING

APPLYING

VALIDATING

DEBUGGING
```

```
REVALIDATING

REVIEWING

APPROVAL

COMPLETED
```

Failure states:

```
FAILED

ROLLBACK

CANCELLED
```

The system should never operate outside the state machine.

Safety Requirements

Mandatory:

- Sandbox isolation
- Git integration
- Structured responses
- Operation validation
- Command restrictions
- Rollback capability

Never trust AI output.

Everything must be verified by Python.

User Experience

Initial interface:

CLI application.

Example:

```
ai-dev /path/to/project
```

User experience:

```
AI Development Agent

Project: MedZone
Framework: Django
Git: Enabled

> Add notification system for students
```

Expected output:

```
Planning...

Relevant Files Found: 12

Generating Changes...

Applying To Sandbox...

Running Validation...

✓ Django Check Passed
✓ Tests Passed
✓ Lint Passed

Reviewing...

Changes Ready

Apply Changes? [Y/N]
```

Recommended Libraries

Core:

- Python 3.12+

Configuration:

- python-dotenv

CLI:

- typer
- rich

Validation:

- pytest
- ruff
- mypy

Data Models:

- pydantic

Standard Library First:

- pathlib
- os
- shutil
- subprocess
- tempfile
- ast
- json
- difflib
- logging

Avoid unnecessary dependencies.

Do NOT introduce:

- LangChain
- LangGraph
- FAISS
- Vector databases
- Agent frameworks

during the initial versions.

Build the orchestration manually first.

Development Roadmap

V1 — File Agent

Features:

- CLI
- AI API integration
- Structured operations
- File creation
- File modification
- Directory management

No autonomy.

V2 — Safe Agent

Features:

- Sandbox
- Git integration
- Diff generation
- Rollback

Focus on safety.

V3 — Validation Agent

Features:

- Test execution
- Django validation
- Ruff
- Mypy
- Error collection

Focus on quality.

V4 — Autonomous Agent

Features:

- Planner
- Coder
- Debugger
- Reviewer
- Retry loop

Focus on automation.

V5 — Multi-Language Agent

Features:

- Django adapter
- Python adapter
- FastAPI adapter
- Node adapter
- React adapter

Focus on framework expansion.

Final Vision

The completed system should function as a local software engineering agent where:

AI provides intelligence.

Python provides control.

Validation provides quality.

Git provides safety.

Sandbox provides isolation.

Context management provides cost efficiency.

The result should be a practical, low-cost, highly controlled AI software engineering platform capable of creating, modifying, debugging, validating, and maintaining real-world software projects while minimizing API usage and maximizing reliability.